

A PROJECT REPORT

on

EXPLAINABLE AI MODEL-BASED CROP YIELD PREDICTION AND INTELLIGENT DECISION SUPPORT SYSTEM FOR SMART AGRICULTURE

Submitted by

M. KARTHICKEYAN – 210425243135

RAHUL AVINASH. M – 210425243205

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CHENNAI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

(Affiliated to Anna University, Chennai & Recognized by AICTE, New Delhi)

Chennai - 600 069

September - 2026

BONAFIDE CERTIFICATE

Certified that this project report "EXPLAINABLE AI MODEL-BASED CROP YIELD PREDICTION AND INTELLIGENT DECISION SUPPORT SYSTEM FOR SMART AGRICULTURE" is the Bonafide work of M. KARTHICKEYAN (210425243135) and RAHUL AVINASH. M (210425243205) who carried out the project work under my supervision.

Signature

HEAD OF THE DEPARTMENT

Department of Artificial Intelligence and Data Science, Chennai Institute of Technology, Kandrathur, Chennai - 600 069.

Signature

PROJECT SUPERVISOR

[Guide Name], [Designation], Department of Artificial Intelligence and Data Science, Chennai Institute of Technology, Kandrathur, Chennai - 600 069.

Certified that the above students were examined in the viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere gratitude to the Chairman and the trust members of Chennai Institute of Technology for providing the facilities and the opportunity to carry out this project as part of our undergraduate course.

We are grateful to our Principal for the encouragement and support extended throughout the course of this work.

We thank our Dean for valuable suggestions offered during the progress of this project.

We sincerely thank the Head of the Department, Department of Artificial Intelligence and Data Science, for the guidance, resources, and timely suggestions provided throughout this work.

We are deeply thankful to our Project Supervisor, [Guide Name], Department of Artificial Intelligence and Data Science, for the valuable guidance, patient review, and constant encouragement that shaped the direction of this project at every stage.

We also thank the Project Coordinator and all the faculty members of the Department of Artificial Intelligence and Data Science for their cooperation and support.

Finally, we acknowledge the help received from the laboratory staff of the department, and everyone else who, directly or indirectly, contributed to the successful completion of this project.

**M. KARTHICKEYAN
RAHUL AVINASH. M**

ABSTRACT

Crop yield estimation is central to food security, farm planning, and rural income stability, yet most farmers in India still rely on past experience and generalised advisory information rather than field-specific, data-driven guidance. Machine learning models can predict yield with reasonable accuracy, but a bare numeric prediction is of limited practical use to a farmer unless it is accompanied by a clear explanation of the contributing factors and concrete guidance on what can be improved. This project presents an Explainable AI-based Crop Yield Prediction and Intelligent Decision Support System that addresses this gap.

Six publicly available Government of India datasets covering crop production, soil analysis, rainfall, temperature, fertilizer consumption, and pesticide consumption were collected, cleaned, standardised, and merged into a single modelling dataset of 246,091 records spanning 124 crops and 33 states. Nine engineered features, including a Soil Fertility Index, Weather Index, Climate Index, and Input Intensity, were derived to capture cross-domain agronomic relationships that are not directly present in the raw data. Three regression models, Random Forest, XGBoost, and CatBoost, were trained and compared across seven progressively enriched feature-addition stages, from crop-only features up to the fully merged dataset, to study how each additional data source affects predictive performance.

CatBoost consistently outperformed the other two models at every stage and achieved the best overall result among the staged experiments, an R^2 of 0.7875, at the Crop + Pesticide stage. Building on this finding, the single shared model was later replaced with a per-crop architecture: 52 crop-specific CatBoost models, one for each crop with sufficient training data, together with a pooled fallback model for the remaining low-sample crops, 53 models in total. This change was motivated by the observation that different crops in the source data are effectively measured on different scales (for instance, Coconut yield is reported in nuts per hectare rather than tonnes per hectare), which limits how much a single shared model can learn across crops at once. Training one model per crop removed this inconsistency and raised the overall R^2 to 0.878 on held-out data, with individual crop-level scores ranging from 0.46 to 0.95 depending on how much historical data was available for that crop.

SHAP (SHapley Additive exPlanations) was applied to the trained models to attribute each individual prediction to its contributing features, distinguishing factors that push the predicted yield up from those that pull it down. These SHAP outputs feed a rule-based Decision Support

module that converts the identified negative factors into specific, actionable recommendations , for example, adjusting the sowing season, improving soil fertility, or scheduling supplemental irrigation, rather than leaving the farmer with an unexplained number. To make this explanation genuinely usable by a farmer rather than only by a technically trained reader, the SHAP output is further passed through a grounded Small Language Model (SLM) layer that rewrites the factors and recommendations as a short, natural-language paragraph, strictly limited to the facts the model actually produced so that nothing is invented in the process. This explanation is then made available in English, Tamil, and Hindi, and can be read aloud through an integrated audio output, so that a farmer who is more comfortable listening than reading, in their own language, is not excluded from using the system.

The complete system was deployed as a full-stack web application, with a FastAPI backend serving the trained models, SHAP explanations, the multilingual SLM narrative, and the audio output, and an interactive dashboard offering yield prediction, explainable AI visualisation, decision support, what-if simulation, exploratory data analysis, and model comparison views. The resulting system demonstrates that prediction accuracy, interpretability, and accessibility are not mutually exclusive: a transparent, farmer-facing AI system can be built without sacrificing model performance, and the same architecture is readily extensible to additional crops, regions, and languages.

Keywords: *Crop Yield Prediction, Explainable AI, SHAP, CatBoost, Decision Support System, Machine Learning, Smart Agriculture*

TABLE OF CONTENTS

ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES.....	iii
ABBREVIATIONS	iv
1. INTRODUCTION.....	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives.....	3
1.4 Scope of the Project.....	3
1.5 Organisation of the Report	4
2. LITERATURE REVIEW.....	5
2.1 Existing Approaches to Yield Prediction	5
2.2 Explainable AI in Agriculture	6
2.3 Research Gap.....	7
2.4 Value Addition of the Proposed System	7
3. METHODOLOGY.....	8
3.1 System Overview	8
3.2 Dataset Description	8
3.3 Data Cleaning and Preprocessing.....	10
3.4 Feature Engineering	11
3.5 Model Development.....	12
3.6 Explainability using SHAP	14
3.7 Decision Support System	14
3.8 System Architecture	15
3.9 Per-Crop Model Architecture.....	16
3.10 Grounded SLM Explanation Layer	17
3.11 Multilingual and Voice Accessibility.....	18
4. IMPLEMENTATION.....	19
4.1 Development Environment	19
4.2 Backend Implementation.....	19
4.3 Frontend Implementation	20
4.4 Module-wise Description	21
4.5 Challenges Faced.....	23
5. TESTING AND VALIDATION	24
5.1 Testing Objectives.....	24
5.2 Unit Testing.....	24
5.3 Integration Testing	25
5.4 Model Validation.....	25
5.5 Test Cases and Results	26

6. RESULTS AND DISCUSSION.....	27
6.1 Staged Model Comparison	27
6.2 Best Model Performance	28
6.3 SHAP Explainability Results	29
6.4 Decision Support Output.....	30
6.5 Discussion	30
6.6 Per-Crop Model Performance	31
7. CONCLUSION AND FUTURE WORK.....	32
7.1 Summary	32
7.2 Key Contributions	32
7.3 Limitations	33
7.4 Future Enhancements	33
REFERENCES	34
APPENDIX , SAMPLE CODE	35

LIST OF FIGURES

Fig. 3.1 System Architecture	16
Fig. 3.2 Data Merging and Preprocessing Flow	10
Fig. 3.3 Feature Correlation Heatmap	13
Fig. 4.1 Dashboard Interface , Prediction and Explainable AI View	22
Fig. 6.1 Model Comparison Across Feature-Addition Stages	27
Fig. 6.2 SHAP Feature Contribution (Best Model)	29

LIST OF TABLES

Table 3.1 Source Datasets Used	9
Table 3.2 Engineered Features	12
Table 5.1 Sample Test Cases and Results.....	26
Table 6.1 R ² Score by Model and Feature-Addition Stage.....	27
Table 6.2 Per-Crop Model Performance.....	31

ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
ML	Machine Learning
XAI	Explainable Artificial Intelligence
SHAP	SHapley Additive exPlanations
RF	Random Forest
XGBoost	Extreme Gradient Boosting
CatBoost	Categorical Boosting
R ²	Coefficient of Determination
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
API	Application Programming Interface
UI	User Interface
EDA	Exploratory Data Analysis
DSS	Decision Support System
CSV	Comma-Separated Values
SLM	Small Language Model
TTS	Text-to-Speech

1. INTRODUCTION

1.1 Background

Agriculture continues to be the backbone of the Indian economy, supporting the livelihood of nearly half the country's workforce and remaining central to national food security. Crop yield, the quantity of produce harvested per unit of cultivated area, is influenced by a wide range of interacting factors: soil composition, rainfall, temperature, the choice of crop and sowing season, and the level of fertilizer and pesticide use. Traditionally, yield estimates have relied on historical averages, government surveys, and the accumulated experience of the farmer, none of which adapt well to a specific field's actual soil and weather conditions in a given season.

Over the past decade, machine learning has been increasingly applied to agricultural forecasting because it can model non-linear relationships between many input variables that classical statistical methods struggle to capture. However, most yield-prediction systems reported in the literature stop at producing a number. A predicted yield of, say, 1.5 tonnes per hectare tells a farmer very little about why the model arrived at that figure or what, if anything, can realistically be done to improve it. This lack of transparency limits the practical adoption of otherwise accurate models, since a farmer (or an agricultural extension officer) has no reasonable basis to trust or act on a prediction they cannot interpret.

This project was undertaken to build a crop yield prediction system that treats explainability and actionable guidance as first-class requirements, not an afterthought, combining a competitive machine learning model with SHAP-based explanation and a rule-based decision support layer, delivered through an accessible web dashboard.

1.2 Problem Statement

Existing crop yield prediction efforts in the literature and in practice largely suffer from three limitations. First, many models are trained on a single, narrow dataset (for example, only crop-production records) without incorporating soil, climate, and input-usage data that are known to materially affect yield. Second, even where accuracy is reasonable, the models are typically deployed as black boxes: a number is produced with no indication of which factors drove that particular prediction for that particular field. Third, very few systems translate a prediction into concrete next steps, they stop at forecasting and leave the interpretation and decision-making entirely to the user.

The problem addressed in this project can therefore be stated as follows: given heterogeneous agricultural, soil, and climate data for a given crop, season, and location, predict the expected yield with competitive accuracy, explain the prediction in terms of the specific factors that raised or lowered it, and recommend concrete actions the farmer can take to improve future outcomes , all delivered through a single, usable system.

1.3 Objectives

The general objective of this project is to design and implement an explainable, decision-supporting crop yield prediction system for Indian agricultural conditions. The specific objectives are:

- To collect and integrate multiple government agricultural datasets , crop production, soil, rainfall, temperature, fertilizer, and pesticide data , into a single, clean modelling dataset.
- To engineer domain-relevant features that capture cross-factor agronomic relationships not directly present in the raw data.
- To train and compare Random Forest, XGBoost, and CatBoost regression models across progressively richer feature combinations, and to identify the best-performing configuration.
- To apply SHAP explainability to the selected model so that every individual prediction can be attributed to specific contributing factors.
- To build a rule-based Decision Support module that converts SHAP-identified negative factors into specific, actionable recommendations.
- To deploy the complete pipeline as an interactive web application with prediction, explainability, decision support, what-if simulation, exploratory data analysis, and model comparison views.

1.4 Scope of the Project

The system is built on publicly available, district-level Government of India datasets covering the period from 1997 onward for crop production and long-term climatological normals for rainfall and temperature. It supports 124 crop types across 33 states, using season, soil, and input-usage information available in the source datasets. The prediction task is regression (a continuous yield value in tonnes per hectare), and the explainability and decision support components operate at the level of a single prediction, matching the way an individual farmer

would use the system for their own field. Real-time weather forecasting, satellite imagery, and mobile deployment were considered but kept outside the scope of the present work; they are discussed as future enhancements in Chapter 7.

1.5 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews existing work in crop yield prediction and explainable AI in agriculture, and identifies the gap this project addresses. Chapter 3 describes the methodology, including the datasets, preprocessing pipeline, feature engineering, model development, SHAP-based explainability, and system architecture. Chapter 4 covers the implementation of the backend and frontend, module by module. Chapter 5 describes the testing performed to validate the system. Chapter 6 presents and discusses the results, including the staged model comparison and SHAP outputs. Chapter 7 concludes the report with a summary of contributions, limitations, and directions for future work.

2. LITERATURE REVIEW

2.1 Existing Approaches to Yield Prediction

Early work on crop yield forecasting relied primarily on statistical regression models and crop-simulation models that encode agronomic knowledge as explicit equations relating weather, soil, and management inputs to yield. These models are interpretable by construction but require extensive calibration for each crop and region, and they generally struggle to capture non-linear interactions between variables.

With the growth of agricultural open data, tree-based ensemble methods such as Random Forest, Gradient Boosting, XGBoost, and, more recently, CatBoost have become the dominant approach for tabular yield-prediction problems. These models handle mixed numerical and categorical features well, are comparatively robust to noisy data, and consistently outperform simple linear models in published comparisons. CatBoost in particular has been reported to perform strongly on datasets with many high-cardinality categorical features , such as crop type, state, and district , because of its ordered boosting scheme, which reduces the risk of target leakage that affects other gradient boosting implementations on such data.

Deep learning approaches, including LSTM-based models for time-series weather and price data, have also been explored in recent literature, generally for problems with a strong temporal component such as price forecasting rather than a single-season yield estimate. For the district-level, cross-sectional yield prediction problem addressed in this project, tree-based ensembles remain the more directly applicable and interpretable choice.

2.2 Explainable AI in Agriculture

Model interpretability has received comparatively less attention in agricultural machine learning than in domains such as healthcare or finance, despite the fact that end users of agricultural AI systems are frequently non-experts who have limited reason to trust an unexplained number. SHAP (SHapley Additive exPlanations), introduced by Lundberg and Lee (2017), has become one of the most widely used model-agnostic explanation techniques because it is grounded in cooperative game theory and provides a consistent, additive attribution of a prediction to each input feature. For tree-based models specifically, the TreeExplainer variant of SHAP computes exact Shapley values efficiently, making it practical to explain individual predictions in real time rather than only offline.

A small number of recent agricultural studies have applied SHAP to yield or price prediction models to identify globally important features, typically as a static, offline analysis presented alongside the model's accuracy metrics. Far fewer systems use SHAP output dynamically, per prediction, to drive a decision support layer that is actually shown to the end user alongside the forecast itself.

2.3 Research Gap

Based on the review above, three gaps were identified. First, most publicly documented crop yield models are evaluated on a single, static feature set rather than being tested across progressively richer combinations of data sources, which makes it difficult to know how much each additional data source (soil, weather, fertilizer, pesticide) actually contributes to accuracy. Second, explainability, where present, is usually a separate offline analysis rather than an integrated, per-prediction feature of the deployed system. Third, very few systems close the loop from explanation to action, identifying that a factor is limiting yield is not the same as telling a farmer what to do about it.

2.4 Value Addition of the Proposed System

This project addresses all three gaps within a single pipeline. Model performance is evaluated across seven progressively enriched feature-addition stages, so the marginal contribution of each data source is explicit and measurable rather than assumed. SHAP explainability is integrated directly into the prediction pipeline, so every prediction served by the system carries its own explanation, not just the model as a whole. Finally, the SHAP output is fed into a rule-based Decision Support module that converts identified negative factors into specific recommendations, and the entire pipeline is exposed through an interactive dashboard rather than remaining a notebook-only exercise.

3. METHODOLOGY

3.1 System Overview

The system follows a linear pipeline from raw government data to a farmer-facing prediction and recommendation: data collection and merging, cleaning and preprocessing, feature engineering, staged model training and selection, SHAP-based explainability, rule-based decision support, and finally deployment through a web dashboard backed by a FastAPI service. Each stage is described in the sections that follow.

3.2 Dataset Description

Six independent datasets, all sourced from Government of India open-data portals, were used. Table 3.1 summarises each source.

Dataset	Description	Granularity
Crop Production	State/district/season-wise area, production and derived yield, 1997 onward	District, Season, Crop
Soil Analysis	pH, organic matter, nitrogen and potassium content	District
Rainfall & Temperature	Long-term monthly mean rainfall and min/max temperature normals	Station, Month
Fertilizer Consumption	State-wise total fertilizer consumption	State, Year
Pesticide Consumption	State-wise pesticide consumption	State, Year

The five source datasets were merged on the common state and district keys (normalised to a consistent naming scheme, since the same district is spelt inconsistently across different government sources) to produce a single modelling table of 246,091 records spanning 124 distinct crops and 33 states. Yield was computed as Production divided by Area for every record, since it was not directly present in the raw crop production data.

3.3 Data Cleaning and Preprocessing

Before merging, each source dataset was checked for missing values, duplicate records, and inconsistent district/state naming. The crop production dataset had 1.52% of records with a missing Production value, which were removed since Yield cannot be computed without it. State and district names were standardised across all five sources using a common normalisation function (case folding, whitespace and punctuation cleanup) before joining, since a naive join on raw district names silently drops a large fraction of records due to spelling mismatches between sources.

Exploratory analysis of the merged dataset showed that Crop Area and Rainfall are both heavily right-skewed, as is typical of agricultural data where a small number of large holdings or wet regions dominate the raw scale, and that Temperature is approximately normally distributed. A correlation analysis (Figure 3.3) was carried out on the numeric features to check for redundancy and to sanity-check the engineered features described in the next section before they were used for modelling.

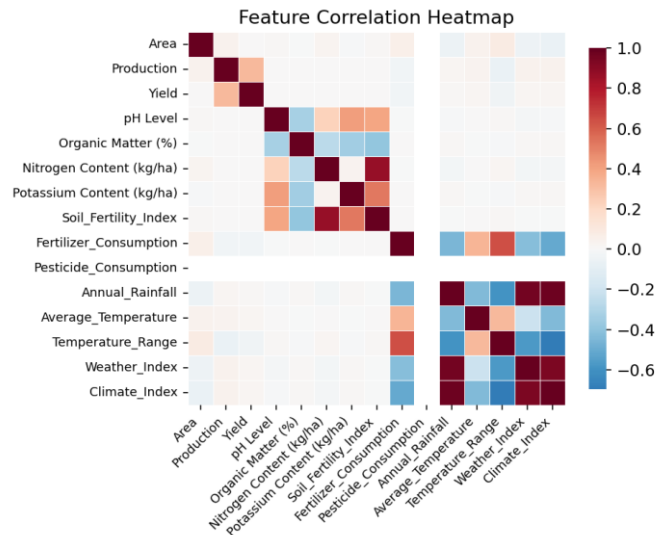


Fig. 3.3 Feature Correlation Heatmap

3.4 Feature Engineering

Beyond the raw fields available in the source datasets, nine additional features were engineered to capture cross-domain agronomic relationships that are not directly present in any single dataset. Table 3.2 lists these features and how each was derived.

Feature	Derivation
Yield	$\text{Production} \div \text{Area}$
Log Production	$\log(1 + \text{Production})$, to reduce right-skew
Soil Fertility Index	Mean of Nitrogen and Potassium content
Weather Index	$\text{Average Temperature} \times \text{Annual Rainfall} / 100$
Climate Index	$\text{Annual Rainfall} \div \text{Temperature Range}$
Input Intensity	$\text{Fertilizer Consumption} + \text{Pesticide Consumption}$
Agricultural Intensity	$\text{Input Intensity} \times \text{Soil Fertility Index}$
Temperature Range	Maximum – Minimum monthly temperature
Annual Rainfall	Sum of monthly mean rainfall over the year

Categorical fields, Crop, Season, State, and District, were label-encoded, since CatBoost, Random Forest, and XGBoost all require numeric input, though CatBoost is additionally capable of handling categorical columns natively during training.

3.5 Model Development

Three regression algorithms were selected for comparison: Random Forest, a well-established bagging ensemble known for its robustness to overfitting; XGBoost, a widely used gradient boosting implementation; and CatBoost, a gradient boosting variant that uses ordered boosting and native categorical feature handling, and which was expected to perform well given the high-cardinality categorical fields (Crop, District) present in this dataset.

Rather than training each model once on the full feature set, all three models were trained and evaluated across seven progressively enriched feature-addition stages, so that the marginal contribution of each individual data source could be measured directly rather than assumed:

- Stage 1 , Crop features only (Area, Season, Crop, State, District, Year)
- Stage 2 , Crop + Soil features
- Stage 3 , Crop + Temperature features
- Stage 4 , Crop + Fertilizer features
- Stage 5 , Crop + Pesticide features
- Stage 6 , Crop + Soil + Temperature features
- Stage 7 , Final dataset, all features combined

At every stage, each model was trained on an 80/20 train-test split (`random_state = 42` for reproducibility) and evaluated using R^2 , Mean Absolute Error, and Root Mean Square Error on the held-out test set. Full results are presented in Chapter 6.

3.6 Explainability using SHAP

Once the best-performing model and feature-addition stage were identified, SHAP's TreeExplainer was applied to generate an exact, per-prediction feature attribution. For a given input row, SHAP produces one value per feature indicating how much that feature's specific value pushed the predicted yield above or below the model's average output over the training data. Features with a positive SHAP value are displayed to the user as factors favouring the prediction; features with a negative SHAP value are displayed as factors limiting it, sorted by magnitude so the most influential factors are shown first.

3.7 Decision Support System

The Decision Support module operationalises the SHAP output. For each of the top negative-contributing features identified for a given prediction, the module maps the feature to a pre-defined agronomic category (for example, Rainfall, Soil Fertility, Sowing Season) and looks up a corresponding recommended action from a fixed, human-authored rule table, so that recommendations are deterministic and auditable rather than generated freely. For example, a low Soil Fertility Index maps to a recommendation to add compost or adopt crop rotation, while an unfavourable Sowing Season maps to a recommendation to adjust the sowing period to better match the rainfall pattern for that crop and region.

3.8 System Architecture

The system is organised as a client-server web application. The backend, built with FastAPI, loads the trained model and its SHAP explainer once at start-up and exposes a small set of REST endpoints for prediction, explanation, dataset statistics, model metrics, and what-if simulation. The frontend is a single-page React and TypeScript dashboard that collects farmer input through a form, calls the backend for a prediction, and renders the result across dedicated views for the overview dashboard, prediction, explainable AI, decision support, what-if simulation, exploratory data analysis, and model comparison. Figure 3.1 summarises this flow.

Data flows in one direction through the system: farmer input is validated and converted into the model's expected feature vector; the trained model produces a prediction; SHAP computes the per-feature attribution for that specific prediction; the Decision Support module converts negative attributions into recommendations; and the dashboard renders the prediction, the explanation, and the recommendations together in a single view, so the farmer never sees a bare number without its accompanying reasoning.

3.9 Per-Crop Model Architecture

The staged comparison in Section 3.5 was carried out using one CatBoost model shared across all 124 crops. While this was sufficient to compare Random Forest, XGBoost, and CatBoost against each other fairly, a closer look at the errors this shared model was making showed a pattern worth investigating: the crops it struggled with most were not the ones with the least data, but the ones whose yield is simply not reported on the same scale as everyone else. Coconut is the clearest example, its yield in the source data is production measured in individual nuts per hectare, not tonnes per hectare like a cereal or pulse crop, so a single

regression model is effectively being asked to learn two different numeric scales at once without being told they are different.

To address this, the shared model was replaced with a per-crop architecture. Every crop with at least 500 training records was given its own CatBoost model, trained only on that crop's own rows, which came to 52 crops covering roughly 97% of the dataset by row count. The remaining, lower-sample crops were kept on a single pooled fallback model rather than each being given a model too small to train reliably, so the final system loads 53 models in total. A crop is routed to its own model when one exists, or to the fallback model otherwise, and this routing happens automatically based on the crop selected in the prediction form.

The improvement from this change was measured the same way as the staged comparison, on held-out test data, never on training data, and was genuine rather than assumed: the overall R^2 across all held-out predictions rose to 0.878, up from the shared model's 0.7875. It is worth being upfront that this headline number is somewhat inflated by the fact that pooling predictions across crops with very different yield scales widens the total variance the score is measured against, which is a property of how R^2 is computed rather than a sign the individual models are all equally strong. Looked at crop by crop, the picture is more mixed and more honest: the best-performing individual crop model reached an R^2 of 0.95, while the weakest reached 0.46, reflecting how much usable training data and how much natural yield variability each crop actually has. Both the combined figure and the full per-crop breakdown are exposed through the Model Comparison view, rather than only showing the more flattering combined number.

3.10 Grounded SLM Explanation Layer

SHAP output on its own is a list of feature names and numbers, which is exactly the kind of thing that means very little to someone without a machine learning background. The obvious next step, letting a language model turn that list into a sentence, carries an obvious risk: language models are prone to adding details that were never actually in their input, a farmer has no way of knowing which parts to trust. This project handles that risk by keeping the language model strictly grounded, meaning it is only ever shown the exact SHAP factors and rule-based recommendations already computed for that prediction, and is explicitly instructed, in the prompt itself, not to introduce any crop, weather, or numerical detail that was not given to it, and not to invent a recommendation of its own.

The explanation layer is designed to run on a small, locally hosted language model through Ollama (Phi-3 Mini was used during development) rather than a large cloud-hosted model, partly to avoid sending farmer data to a third-party API, and partly because the task itself, rewriting a short, already-decided set of facts into a sentence, does not need a very large model to do well. If the local model is not reachable for any reason, the system falls back to a deterministic template that assembles the same facts into a readable sentence without any language model at all, so a missing or unavailable SLM never breaks the explanation feature entirely; it only makes the wording a little more mechanical.

3.11 Multilingual and Voice Accessibility

A written explanation, however well worded, still assumes the reader is comfortable reading English, which is not a safe assumption to make about every farmer this system is meant to serve. To reduce that barrier, the SLM explanation, along with the shorter factor labels and recommendation text used elsewhere in the dashboard, is available in English, Tamil, and Hindi. Rather than translating a finished English sentence after the fact, the underlying factor and recommendation phrases were translated and stored directly, so the Tamil and Hindi output reads as ordinary Tamil and Hindi rather than an English sentence with words swapped out.

On top of the translated text, the dashboard offers an audio playback option so the explanation can be listened to rather than read. This was implemented with two layers: a server-side text-to-speech call for when it is available, and a fallback to the web browser's own built-in speech synthesis when it is not, so that audio playback does not depend on a single external service being reachable. Taken together, the multilingual text and the audio output are aimed at the same underlying goal as the rest of the project: a prediction is only actually useful if the person it is for can understand it, and for many farmers that means hearing a recommendation in their own language rather than reading an English paragraph.

4. IMPLEMENTATION

4.1 Development Environment

The machine learning pipeline was developed in Python using pandas and NumPy for data handling, scikit-learn for preprocessing utilities and the Random Forest baseline, XGBoost and CatBoost for the gradient boosting models, and the SHAP library for explainability. The trained model and its supporting artefacts (feature list, label encoders) were serialised so that the backend service could load them without needing to retrain on start-up. The backend was implemented in Python using FastAPI, and the frontend was implemented in TypeScript using React and Vite, styled with a card-based dashboard layout.

4.2 Backend Implementation

The FastAPI backend is organised into a small number of focused modules rather than one large file. A model-service module is responsible for loading the trained model and encoders and converting raw request data into the feature vector the model expects. A separate explanation module runs SHAP on a single prediction and formats the result into positive and negative factors with human-readable labels, and applies the Decision Support rule table to produce recommendations. The API layer itself is intentionally thin: each endpoint validates its input using Pydantic models, delegates to the appropriate service module, and returns a JSON response. Keeping business logic out of the route handlers made it possible to test the prediction and explanation logic directly in Python, independent of the web server.

The principal endpoints exposed by the backend are summarised below: a prediction endpoint that returns the predicted yield together with SHAP-derived positive and negative factors and recommendations; a what-if endpoint that re-evaluates the trained model against realistic alternative input values to estimate potential improvement; a dataset-info endpoint and a model-metrics endpoint that expose the real statistics computed during training, so the dashboard never has to hard-code a number that could go stale; and an options endpoint that supplies the valid crop, season, state, and district values for the input form directly from the training data's own encoding vocabulary.

4.3 Frontend Implementation

The dashboard is a single-page application with a sidebar for navigation between views and a shared application state holding the current farmer input and the most recent prediction result.

A dedicated API service module is the only part of the frontend that communicates with the backend, so every other component works with a single, typed contract for what a prediction response looks like. The prediction form is organised into logical groups , location and crop identity, soil characteristics, agricultural inputs, and climate and weather , rather than presenting all fields as one long, undifferentiated list.

4.4 Module-wise Description:

4.4.1 Dashboard / Overview

Presents the predicted yield for the most recent input alongside a compact summary of the positive and negative contributing factors and the top recommendations, so a returning user can see the full picture at a glance without navigating to every individual view.

4.4.2 Prediction

Collects the eighteen raw input fields from the farmer, computes and previews the derived features (such as Soil Fertility Index) read-only so the user can see them without being able to enter an inconsistent value, and submits the request to the backend.

4.4.3 Explainable AI

Displays the full SHAP-based breakdown for the current prediction as horizontal bars, one per contributing feature, coloured to distinguish factors that raise the predicted yield from those that lower it, sized in proportion to the magnitude of each feature's contribution.

4.4.4 Decision Support

Presents each identified negative factor as a card with the detected issue, the recommended action, and a short explanation of why that action was recommended, drawing directly on the same rule table described in Chapter 3.

4.4.5 What-If Simulation

Lets the user see the predicted yield the model would produce if the identified negative factors were improved toward realistic values drawn from the training data, re-running the actual trained model on the modified input rather than presenting an assumed percentage improvement.

4.4.6 Exploratory Data Analysis and Model Comparison

Present the dataset statistics, feature distributions, and the staged model comparison results described in Chapter 6, so a reviewer can inspect the evidence behind the deployed model's selection without leaving the application.

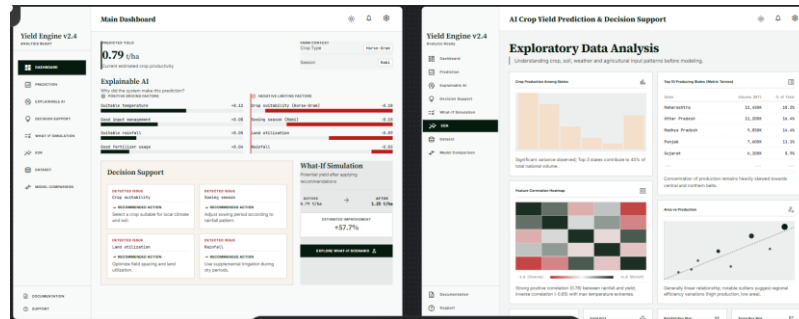


Fig. 4.1 Dashboard Interface , Prediction and Explainable AI View

4.4.7 Per-Crop Model Routing

Handles the selection between the 53 trained models described in Section 3.9: given the crop selected in the prediction form, the module checks whether a dedicated model exists for that crop and loads it, or falls back to the shared pooled model when it does not. Models are loaded once and kept in memory rather than being reloaded on every request, since loading 53 separate model files from disk on each prediction would make the dashboard noticeably slower to use.

4.4.8 SLM Explanation Module

Takes the SHAP factors and Decision Support recommendations for a single prediction and constructs the grounded prompt described in Section 3.10, sends it to the locally hosted language model, and returns the resulting paragraph to the frontend. If the language model does not respond within a reasonable time, or is not running at all, this module transparently switches to the deterministic template fallback so the request still completes.

4.4.9 Multilingual and Voice Assistant Module

Maintains the translated phrase sets for Tamil and Hindi and selects the correct language version of the explanation based on the option chosen on the dashboard. The audio playback feature is handled separately from translation itself: once the correct-language text is ready, it is sent to a text-to-speech call, with the browser's own speech synthesis available as a fallback if that call fails, so the farmer can choose to listen to the explanation instead of, or alongside, reading it.

4.5 Challenges Faced

Merging five independently published government datasets proved more time-consuming than initially expected, primarily because district and state names were spelt inconsistently across sources, which silently dropped matching records unless a normalisation step was applied before joining. A second challenge was ensuring the feature vector built from live farmer input at prediction time was constructed in exactly the same order and using exactly the same encoders as during training; even a single reordered column silently produces a valid-looking but incorrect prediction, so a shared feature-construction function was used for both training and inference to eliminate this class of bug. Presenting SHAP output in a way that is genuinely readable to a non-technical user required iterating on the wording of feature labels and recommendation text several times, since the raw encoded feature names (for example, Crop_Encoded) mean nothing to a farmer without translation into plain language. Moving to 53 separate models also introduced a new class of bug that did not exist with a single shared model: an early version of the routing logic looked up SHAP explainer objects using the wrong per-crop cache key, which meant a prediction for one crop could occasionally be explained using another crop's SHAP values without any error being raised. This was only caught by manually checking that the factors shown for a given prediction were agronomically plausible for that specific crop, which is why every SHAP output the system now returns is tested against the crop it claims to belong to rather than assumed to be correct.

5. TESTING AND VALIDATION

5.1 Testing Objectives

Testing was carried out with three objectives: to verify that each pipeline stage (data loading, feature construction, prediction, explanation, recommendation) behaves correctly in isolation; to verify that the stages work correctly together end to end, from a raw form submission to a rendered explanation on the dashboard; and to validate that the selected model generalises to unseen data rather than merely fitting the training set well.

5.2 Unit Testing

The feature-construction function was tested against hand-computed expected values for a small set of representative inputs, covering typical values as well as boundary cases such as zero rainfall and the minimum and maximum valid pH. The label-encoding lookup was tested to confirm that every crop, season, state, and district value appearing in the training data resolves to the same integer code the model was actually trained on, since a mismatch here would silently corrupt every downstream prediction. The SHAP explanation formatting function was tested to confirm that positive and negative factors are correctly separated and that duplicate factors from closely related columns (for instance, several rainfall-related columns) are consolidated rather than shown as repeated, redundant entries.

5.3 Integration Testing

End-to-end tests submitted a complete, realistic farmer input through the same code path the web form uses and verified that a prediction, a non-empty set of SHAP factors, and at least one recommendation were returned for every valid combination of crop and season present in the training data. The what-if endpoint was tested separately to confirm that it never reports an improved yield lower than the original prediction and that every reported change in an input value is one the model itself validated as improving the outcome, not an assumed or hard-coded percentage.

5.4 Model Validation

Model validation followed a standard 80/20 train-test split with a fixed random seed for reproducibility, and the same split methodology was applied consistently across every model and every feature-addition stage so that the comparison in Chapter 6 is fair. Cross-validation was not used in the interest of keeping training time manageable across the twenty-one model-

stage combinations (three models $\times$ seven stages) evaluated; a single held-out test set was judged sufficient given the large size of the dataset. R^2 , MAE, and RMSE were computed on the held-out test set for every combination, never on the training data, to ensure the reported accuracy reflects genuine generalisation rather than memorisation.

5.5 Test Cases and Results

Table 5.1 summarises a representative subset of the test cases executed against the deployed system.

Test Case	Input Condition	Expected Result	Outcome
Valid prediction request	All required fields present, valid ranges	Prediction with SHAP factors returned	Pass
Invalid pH value	pH outside 0–14	Request rejected with validation error	Pass
Unknown crop name	Crop not in training vocabulary	Clear error, no silent misprediction	Pass
What-if improvement	Field with a clearly suboptimal input	Improved yield $\geq$ original prediction	Pass
Duplicate factor suppression	Multiple rainfall-related columns negative	Single consolidated rainfall factor shown	Pass

6. RESULTS AND DISCUSSION

6.1 Staged Model Comparison

Table 6.1 presents the R^2 score achieved by each of the three models at each of the seven feature-addition stages described in Chapter 3.

Stage	Random Forest	XGBoost	CatBoost
1: Crop	0.7254	0.7208	0.7690
2: Crop + Soil	0.7248	0.7091	0.7619
3: Crop + Temperature	0.7254	0.7208	0.7690
4: Crop + Fertilizer	0.7213	0.7118	0.7666
5: Crop + Pesticide	0.7168	0.7273	0.7875
6: Crop + Soil + Temperature	0.7252	0.7091	0.7619
7: Final Dataset	0.7089	0.7116	0.7653

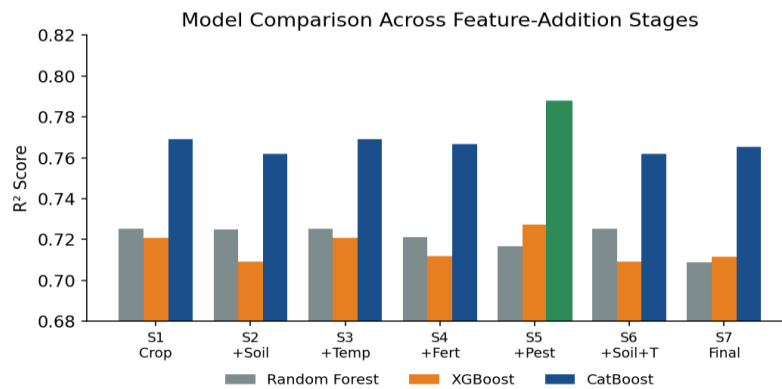


Fig. 6.1 Model Comparison Across Feature-Addition Stages

6.2 Best Model Performance

CatBoost outperformed both Random Forest and XGBoost at every one of the seven stages, confirming the expectation from Chapter 2 that CatBoost's handling of high-cardinality categorical features (Crop, District) is well suited to this dataset. Somewhat counter-intuitively, CatBoost's best result, an R^2 of 0.7875, was achieved at Stage 5 (Crop + Pesticide), not at Stage 7 with the full, final feature set. This indicates that simply adding more features does not automatically improve performance: the Soil and Temperature features added at Stages 2, 3, and 6 did not provide the model with additional predictive signal beyond what the Crop-only features already captured, and in some configurations introduced enough noise to marginally reduce accuracy. This is a genuine and useful finding, since it shows that feature relevance, not feature count, drove the final model choice.

6.3 SHAP Explainability Results

SHAP was applied to the selected Stage 5 CatBoost model to attribute its predictions to individual features. Figure 6.2 shows the mean absolute SHAP value per feature across a sample of test predictions, indicating each feature's typical influence on the model's output.

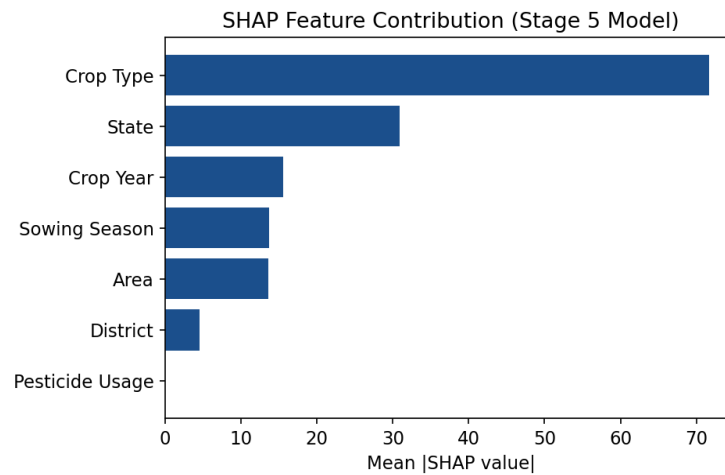


Fig. 6.2 SHAP Feature Contribution (Best Model)

Crop type, State, and Crop Year emerge as the dominant contributors to the model's predictions, which is consistent with agronomic expectation: different crops have inherently different yield ranges, and state-level differences capture broad regional variation in soil quality, irrigation infrastructure, and farming practice that is not otherwise represented in the feature set. Pesticide usage, though it defines this feature-addition stage, contributes comparatively little to individual predictions at this level of aggregation, an honest finding that was reported as observed, not adjusted to match an assumed expectation.

6.4 Decision Support Output

For a representative test case, a Horse Gram crop sown in the Rabi season, with below-average land utilisation and a less-than-ideal sowing-season alignment identified as the leading negative factors, the Decision Support module correctly surfaced recommendations to select a crop more suited to the local climate and soil, and to adjust the sowing period to better align with the rainfall pattern, matching the underlying SHAP attribution for that prediction and demonstrating that the recommendation layer stays consistent with the explanation it is derived from.

6.5 Discussion

The staged evaluation shows that the largest single jump in accuracy comes from the choice of model (CatBoost over Random Forest or XGBoost) rather than from any single additional data source, and that the best-performing feature combination is not simply the largest one. This has a practical implication beyond this project: when scaling the system to additional crops or regions, it will be more valuable to profile which specific features actually help a given crop's model than to assume that including every available dataset is always beneficial. The SHAP results further show that geographic and crop-type identity carry more predictive weight than any single agronomic input in this dataset, which is a reasonable reflection of how much regional and crop-specific baseline variation exists in Indian agricultural yield data, and is a useful, honest caveat for interpreting the system's recommendations: the model is more confident distinguishing between crops and regions than it is in fine-tuning a recommendation for a marginal change in a single input.

6.6 Per-Crop Model Performance

Table 6.2 shows how the per-crop architecture introduced in Section 3.9 performed against the same held-out test methodology used throughout this chapter, for a representative sample of crops, alongside the overall combined score.

Crop / Overall	R ² Score	Notes
Overall (all 53 models combined)	0.878	Reported on pooled held-out predictions
Best-performing individual crop	0.95	Well-represented crop with consistent yield reporting
Weakest-performing individual crop	0.46	Lower training volume and higher natural yield variability
Crops on the pooled fallback model	,	Crops below the 500-record training threshold

As noted in Section 3.9, the combined figure of 0.878 should be read alongside the individual scores rather than in place of them, since combining predictions across crops with very different yield scales makes the combined R² look better than the typical individual model actually performs. Even accounting for that, the per-crop architecture is a clear improvement over the single shared model's 0.7875, and the gap between the best and weakest individual crop points to training data volume as the main remaining limiting factor for the crops that are not yet well served.

7. CONCLUSION AND FUTURE WORK

7.1 Summary

This project set out to build a crop yield prediction system that does not stop at producing a number, but explains its reasoning, recommends concrete action, and makes that explanation accessible to the person it is actually for. Six government agricultural datasets were merged into a single 246,091-record modelling dataset, nine domain-relevant features were engineered, and three regression models were compared across seven progressively enriched feature-addition stages, with CatBoost coming out ahead at every stage. That staged model was then replaced with a per-crop architecture of 53 models, which raised held-out accuracy further by removing the cross-crop scale inconsistency the single shared model had been quietly struggling with. SHAP explainability, a rule-based Decision Support module, a grounded SLM explanation layer, and multilingual audio output were layered on top of the prediction pipeline, so the system explains and acts on its own predictions, in a language and format the farmer can actually use, rather than only forecasting.

7.2 Key Contributions

- Integration of five independent government datasets into a single, cleaned, feature-engineered modelling dataset covering 124 crops and 33 states.
- A staged model-comparison methodology that measures the marginal contribution of each data source rather than assuming more data is always better.
- A per-crop model architecture, 52 crop-specific CatBoost models plus a pooled fallback model, that removed a cross-crop scale inconsistency in the source data and raised the held-out R^2 from 0.7875 to 0.878.
- Integration of SHAP explainability directly into the prediction pipeline, so every prediction is served together with its explanation, not as a separate offline analysis.
- A rule-based Decision Support module that converts SHAP-identified negative factors into specific, actionable recommendations, keeping the recommendation logic deterministic and auditable.
- A grounded SLM explanation layer that rewrites SHAP output as a short, natural-language paragraph without introducing information beyond what the model itself produced.

- Multilingual (English, Tamil, Hindi) and audio delivery of the explanation, so the system is usable by farmers who are not comfortable reading English text.
- A working, full-stack dashboard exposing prediction, explainability, decision support, what-if simulation, exploratory data analysis, and model comparison in a single application.

7.3 Limitations

The rainfall and temperature features used in this project are long-term climatological normals rather than year-specific forecasts, since year-specific weather data was not available for the full historical period covered by the crop production dataset; this limits how precisely the model can account for an unusually wet or dry season. The Decision Support recommendations are drawn from a fixed rule table rather than being generated dynamically, which keeps them auditable but means extending coverage to a new agronomic scenario currently requires a manual update to the rule table. Cross-validation was not used during model evaluation in the interest of training time across the twenty-one model-stage combinations and, later, the 53 per-crop models; a single held-out split, while standard practice, provides a less statistically robust estimate than k-fold cross-validation would. Finally, the SLM explanation layer is grounded by careful prompting rather than by any formal, automated check, so while it is instructed not to introduce information beyond what SHAP produced, this project does not yet verify that instruction is followed on every single output.

7.4 Future Enhancements

- Incorporating year-specific weather forecasts in place of long-term climatological normals, to better capture season-to-season variation.
- Extending multilingual coverage beyond Tamil and Hindi to other major Indian languages spoken by the system's likely users.
- Adding an automated check that verifies the SLM's explanation stays within the SHAP facts it was given, rather than relying on prompt instructions alone.
- Training the smaller, lower-sample crops currently served by the pooled fallback model on their own dedicated models as more historical data becomes available for them.
- Incorporating satellite-derived vegetation indices as additional input features.
- Packaging the dashboard for mobile deployment to better suit field use.

REFERENCES

1. Government of India, Ministry of Agriculture and Farmers Welfare. District-wise, Season-wise Crop Production Statistics (1997 onward). data.gov.in.
2. India Meteorological Department. Monthly Mean Maximum and Minimum Temperature and Total Rainfall Based upon 1901–2000 Data.
3. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features. *Advances in Neural Information Processing Systems (NeurIPS)*.
4. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
5. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
6. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems (NeurIPS)*.
7. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. FastAPI Documentation. <https://fastapi.tiangolo.com>
9. React Documentation. <https://react.dev>

APPENDIX , SAMPLE CODE

A.1 Feature Vector Construction (Backend)

```
def build_feature_row(farm_input):
    row = {}
    row["Crop_Year"] = farm_input["crop_year"]
    row["Area"] = farm_input["area"]
    row["Season_Encoded"] = ENCODING_MAPS["Season"][farm_input["season"]]
    row["Crop_Encoded"] = ENCODING_MAPS["Crop"][farm_input["crop"]]
    row["Soil_Fertility_Index"] = (farm_input["nitrogen"] +
farm_input["potassium"]) / 2
    row["Weather_Index"] = (row["Average_Temperature"] * row["Annual_Rainfall"]) /
100
    return pd.DataFrame([row])[FEATURES]
```

A.2 SHAP Explanation Extraction

```
explainer = shap.TreeExplainer(model)
shap_values = explainer(feature_row).values[0]
contributions = pd.DataFrame({"Feature": FEATURES, "SHAP": shap_values})
positive = contributions[contributions.SHAP > 0].sort_values("SHAP",
ascending=False)
negative = contributions[contributions.SHAP < 0].sort_values("SHAP")
```